

Security Assessment, Remediation and OpenID Connect Integration for a MERN-Stack Doctor Appointment Booking System

Nethmina W.P.R.
Faculty of Computing

Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
IT22253958

Appuhami M.N.H.
Faculty of Computing

Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
IT22140852

Madurapperuma H.A.S.I.
Faculty of Computing

Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
IT22230942

Aron Charles J.
Faculty of Computing

Sri Lanka Institute of Information
Technology
Malabe, Sri Lanka
IT22203380

Abstract—This report presents the security assessment and hardening of Prescripto, a MERN-stack (MongoDB, Express, React, Node.js) doctor appointment booking system with patient, doctor and administrator roles. Using manual white-box code review, software composition analysis, dynamic testing against a running instance and automated regression tests, we identified twenty distinct vulnerabilities spanning nine of the OWASP Top 10:2021 categories. They include committed production credentials, an administrator token that embedded the administrator password, a four-character JWT signing secret, NoSQL operator injection, account pre-hijacking through social login, a rate-limit bypass via spoofed proxy headers and a race condition that allowed double booking. All twenty were fixed in individually documented commits and verified by 41 automated security tests and a 23-check end-to-end test against a containerised deployment. We also implemented Google Sign-In based on OpenID Connect, with server-side ID-token validation and a server-issued, single-use nonce that prevents token replay. Finally, we added a continuous security pipeline and a hardened Docker deployment, and we discuss residual risks and the secure-development practices that would have prevented these flaws.

Index Terms—secure software development, web application security, OWASP Top 10, OpenID Connect, JSON Web Token, NoSQL injection, DevSecOps, container security.

I. INTRODUCTION

Web applications that manage medical appointments process personal data such as names, telephone numbers, addresses and dates of birth, and they expose authentication and booking workflows directly to the Internet. Their typical weaknesses are well documented by the OWASP Top 10 [1] and the Common Weakness Enumeration (CWE) [2], yet they continue to appear in many projects where functionality is prioritised over security.

This work was carried out for the SE4030 Secure Software Development module, which requires identifying and fixing at least seven distinct vulnerabilities in an existing application and implementing a feature based on OAuth 2.0 or OpenID Connect. We selected Prescripto, a MERN-stack doctor appointment booking system [3]. It offers sufficient scope: three trust levels (patient, doctor, administrator), token-based authentication, file uploads to a third-party media service, and 24 REST endpoints. It is not a deliberately vulnerable teaching

application. The hardened version, including the full commit history, is published separately [4].

The main contributions of this work are:

- a two-round assessment that identified 20 distinct vulnerabilities, each mapped to the OWASP Top 10:2021 and CWE and supported by before-and-after evidence;
- remediation of all 20 vulnerabilities, each in an individually documented commit;
- Google Sign-In based on OpenID Connect with server-side token validation, nonce-based replay protection and safe account linking;
- an automated security regression suite, a continuous security pipeline and a hardened container deployment; and
- an analysis of residual risks and of the development practices that would have prevented the vulnerabilities.

Section II describes the system, Section III the methodology and Section IV the vulnerabilities and their fixes. Section V lists issues that were not fixed. Section VI presents the OpenID Connect implementation, Section VII the verification, Section VIII the secure deployment and Section IX the preventive practices. Sections X and XI give the individual contributions and the conclusion.

II. SYSTEM OVERVIEW

A. Architecture

Prescripto consists of three deployable parts, summarised in Table I. The back end is an Express 4 REST API that uses Mongoose 8 models for users, doctors and appointments, JSON Web Tokens (JWT) [5] for sessions, bcrypt for password hashing and Multer for multipart uploads, which are forwarded to Cloudinary. Two React 18 single-page applications built with Vite provide the patient interface and a combined administrator and doctor panel. Figure 1 shows the hardened deployment described in Section VIII.

B. Assets, Trust Boundaries and Threat Actors

The assets are patient personal data and appointment records, doctor and administrator accounts, the administrator's ability to create doctors and view all appointments, and the secrets that protect them (database credentials, the JWT signing key and third-party API keys). The main trust boundaries are browser to API, API to external services (MongoDB, Cloudinary, Google) and developer to repository. We considered three threat actors: an unauthenticated Internet attacker; an authenticated patient or doctor attempting to

This report was prepared for the SE4030 Secure Software Development module, Sri Lanka Institute of Information Technology, September 2026. Source code of the hardened system:
<https://github.com/RVNethmina/SE4030-secure-appointment-system>.

exceed their privileges; and anyone able to read the source repository.

TABLE I
APPLICATION COMPONENTS

Component	Technology	Responsibility
backend	Node.js, Express 4.22, Mongoose 8.24, jsonwebtoken, bcrypt, Multer	REST API (24 endpoints): authentication, booking, profiles, uploads
frontend	React 18.3, Vite 6.4, axios, @react-oauth/google	Patient app: doctors, booking, profile, Google sign-in
admin	React 18.3, Vite 6.4, axios	Administrator panel (doctors, all appointments) and doctor panel
External	MongoDB, Cloudinary, Google Identity Services	Persistence, image hosting, OpenID Connect provider

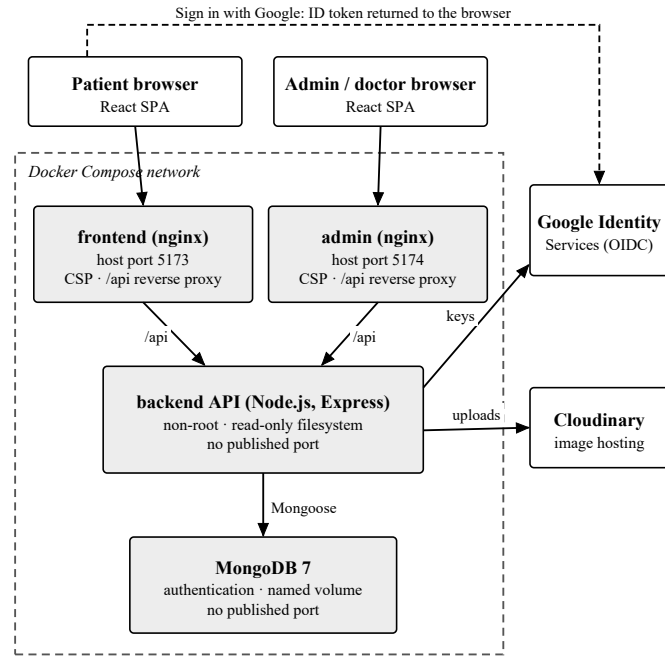


Fig. 1. Hardened deployment. Only the two nginx containers publish ports; they serve the single-page applications and proxy /api requests to the API, which alone can reach MongoDB.

C. Baseline

The unmodified application at commit 2c2bad8 (February 2025) was imported as the first commit of the hardened repository. Live external credentials were redacted during import, but the weak secrets were kept so that the corresponding findings remain reproducible. Every later change is a separate, descriptively commented commit.

III. METHODOLOGY

A. Approach

The assessment was performed in two rounds. Round 1 (July 2026) reviewed the original application and fixed ten vulnerabilities. Round 2 (September 2026) reviewed the hardened code again, including the round 1 fixes themselves, and found ten further vulnerabilities. Two of them (V12 and V13) had been missed or even introduced by round 1 fixes, which shows the value of reviewing security changes as critically as feature code.

B. Techniques and Tools

Table II lists the techniques used. Manual white-box review covered every controller, middleware, route, model and configuration file of the API, and the authentication and session handling of both clients. The review was guided by the authentication, session management, access control and validation requirements of OWASP ASVS [6]. Dynamic tests sent crafted requests to a locally running instance, and each confirmed vulnerability was turned into an automated regression test.

TABLE II
ASSESSMENT TECHNIQUES AND TOOLS

Activity	Technique / tool	Scope
White-box review	Manual review guided by OWASP ASVS [6]	All API code; client authentication and logging
Composition analysis	npm audit [7]; Dependabot	backend, frontend and admin packages
Secret detection	Git history inspection; gitleaks [8]	Repository history and every push
Static analysis	GitHub CodeQL, security-extended queries [9]	All JavaScript code, in CI
Dynamic testing	curl against a running API	Injection, forged tokens, CORS, uploads, proxy headers
Regression and E2E	node:test + supertest; Node.js E2E script	41 tests; 23 checks against Docker

C. Severity Rating

Each vulnerability was rated Critical, High, Medium or Low from its exploitability (for example, whether an attacker must be authenticated) and its impact on confidentiality, integrity and availability. The ratings follow the reasoning of the CVSS v3.1 base metrics [10] but are qualitative, not numeric scores.

IV. VULNERABILITY ASSESSMENT AND REMEDIATION

Table III summarises the 20 vulnerabilities. Fig. 2 shows their distribution across the OWASP Top 10:2021 categories: nine of the ten categories are represented, and identification and authentication failures (A07) dominate. The remainder of this section describes each vulnerability, its impact, the fix and how the fix was verified.

TABLE III
SUMMARY OF IDENTIFIED VULNERABILITIES

ID	Vulnerability	OWASP	CWE	Severity
V1	Secrets committed to version control	A02, A07	798, 312	Critical
V2	Forgeable admin token containing the admin password	A07	287, 522	Critical
V3	Non-expiring JWTs with a 4-character secret	A02, A07	613, 326	Critical
V4	No brute-force protection on login	A07	307	High
V5	Wildcard CORS and missing security headers	A05	942, 693	Medium
V6	NoSQL operator injection and user enumeration	A03	943, 204	High
V7	Unrestricted file upload	A04	434, 400	Medium
V8	Passwords in logs; verbose error messages	A09, A04	532, 209	Medium
V9	Weak password policy	A07	521	Medium
V10	Vulnerable third-party components	A06	1104	High
V11	Missing role enforcement; unpinned JWT algorithm	A01	863, 347	Medium

ID	Vulnerability	OWASP	CWE	Severity
V12	Upload processed before authentication	A04, A07	306, 434	High
V13	Rate-limit bypass via X-Forwarded-For	A07	348, 307	High
V14	Account pre-hijacking; login oracles	A07	287, 204, 208	High
V15	Weak admin credential accepted at boot	A07	521, 208	Medium
V16	Double-booking race; booking logic flaws	A04	362, 20, 840	Medium
V17	Mass assignment in profile updates	A08	915, 20	Medium
V18	Replayable Google ID tokens (no nonce)	A07	294	Medium
V19	Passwords, tokens and PII in browser console	A09	532, 359	Medium
V20	Rejected sessions not cleared by clients	A07	613	Low

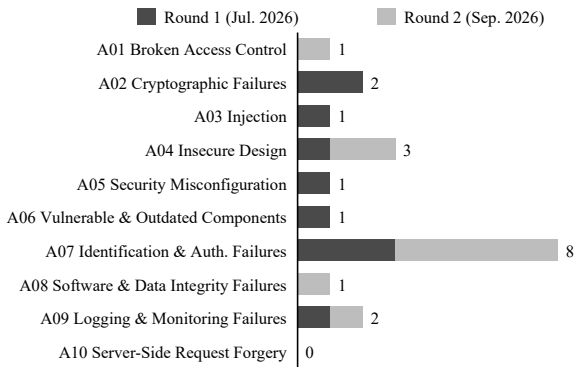


Fig. 2. Number of vulnerabilities per primary OWASP Top 10:2021 category, by assessment round.

A. Round 1 Findings

V1) Secrets committed to version control: The original repository tracked `backend/.env`, which contained the MongoDB Atlas connection string with its password, the Cloudinary API key and secret, a Razorpay key, the administrator password (`qwerty123`) and the JWT signing secret (`RBRO`). Anyone able to read the repository gained full database access and control of the media store, and could forge sessions (V2, V3). Because git keeps history, deleting the file later does not end the exposure.

Remediation: All `.env` files were removed from version control, `.gitignore` now excludes them, and `.env.example` templates document the required variables without values. The credentials that were exposed must be rotated by their owners (Section V), and a `gitleaks` job now fails any push that contains a credential (Section VII).

V2) Forgeable administrator token: The administrator login compared the submitted credentials with environment variables and issued `jwt.sign(email + password, secret)`, so the token payload was the concatenated email address and password. The middleware accepted any token whose payload equalled that string (Listing 1). JWT payloads are only Base64url-encoded, so anyone who obtained an administrator token, for example from browser storage or a proxy log, could read the administrator password in cleartext. The token never expired, and anyone who knew the committed secret (V1, V3) could create one offline. The administrator

role can create doctor accounts and read every patient's appointments.

```
// Before: adminController.js and authAdmin.js
const token = jwt.sign(email + password,
  process.env.JWT_SECRET);
if (token_decode !== process.env.ADMIN_EMAIL
  + process.env.ADMIN_PASSWORD) { ... }

// After
const token = signAccessToken({ role: "admin", email });
if (token_decode.role !== "admin")
  return res.status(403).json({ success: false, ... });
```

Listing 1. Administrator token issuance and check, before and after.

Remediation: Tokens now carry a role claim and an expiry, and the middleware authorises requests only on the verified role claim, returning HTTP 401 or 403 on failure. Credentials are compared in constant time (strengthened in V15). A regression test confirms that a token built with the original scheme is rejected.

V3) Non-expiring tokens with a four-character secret: User and doctor tokens were created with `jwt.sign({ id }, secret)` and no expiry, so a stolen token remained valid forever. The HMAC secret had four characters and can be brute-forced offline from any single token within seconds; it had also been committed (V1). Knowing the secret allows forging a token for any user or doctor, which means taking over any account.

Remediation: A new module `utils/token.js` performs all signing with a mandatory expiry (`JWT_EXPIRES_IN`, default one day), reads bearer tokens consistently, and provides `assertJwtSecret()`, which stops the server at start-up if the secret has fewer than 32 characters. In round 2, verification was restricted to HS256, as recommended by the JWT best current practices [11]. Tests confirm that tokens signed with the old secret, expired tokens and tokens using other algorithms are rejected, and that the server refuses to start with the old secret.

V4) No brute-force protection: The login endpoints of all three roles and the registration endpoint accepted unlimited requests. This enabled online password guessing and credential stuffing at machine speed, made worse by the enumeration oracle described in V6.

Remediation: `express-rate-limit` now allows 10 authentication requests per client IP address per 15 minutes (login, registration and Google sign-in) and 300 requests per 15 minutes to the API as a whole. Excess requests receive HTTP 429. Round 2 later found that this limit could be bypassed (V13).

V5) Wildcard CORS and missing security headers: `app.use(cors())` returned `Access-Control-Allow-Origin: *` on every endpoint, and no security headers were sent: no content-type sniffing protection, framing protection, HSTS or Content Security Policy, while `X-Powered-By` revealed the framework. Because the API reads tokens from custom headers rather than cookies, a malicious site could not ride a victim's session. The impact was therefore a loss of defence in depth: any origin could script the API with a stolen token and read the responses, and the browser applied no protective policies.

Remediation: CORS is now restricted to the origins listed in `ALLOWED_ORIGINS`, and other origins receive HTTP 403. `helmet` sets security headers, and a central error handler returns only generic messages. In the Docker deployment,

nginx also sends a strict Content Security Policy (Section VIII).

V6) NoSQL operator injection and user enumeration: The login handlers passed the request body directly to `findOne({email})`. Because the JSON parser accepts nested objects, an email value such as `{"$regex": "^a"}` turns the lookup into an operator query [12]. The patient login returned different messages depending on whether an account matched (Listing 2). This was a yes/no oracle: an attacker could recover every registered email address character by character, and try passwords against accounts whose addresses they did not know. `bcrypt` rejects non-string passwords, which prevented a full authentication bypass, but the resulting exception text was returned to the client (V8).

```
POST /api/user/login
{"email": {"$regex": "^a"}, "password": "x"}

-> "User doesn't exist"      no address starts with "a"
-> "Invalid Credentials!"   an address starts with "a"
```

Listing 2. Blind extraction of registered email addresses (original code).

Remediation: `express-mongo-sanitize` removes keys that start with `$` or contain a dot. Explicit string type checks run before any query, and all login failures return the same message. Round 2 also made the response time independent of whether the account exists (V14).

V7) Unrestricted file upload: Multer saved uploads in the operating system's temporary directory under the file name chosen by the client, with no limit on size, number or type. An attacker could exhaust the disk with large files. Two uploads with the same name also overwrote each other before being sent to Cloudinary, so one person's photograph could end up on another person's profile or doctor record. Multer strips directory components from file names, so path traversal was not possible.

Remediation: Uploads are now written to a dedicated directory under random 128-bit names, only JPEG, PNG and WEBP types and extensions are accepted, and requests are limited to one file of at most 2 MB. Round 2 added authentication before upload parsing, verification of the file's actual content, and deletion of temporary files (V12).

V8) Passwords in logs and verbose error messages: The add-doctor handler wrote the complete request body, including the new doctor's plaintext password, to the server log. Every error handler also returned the raw exception message to the client, revealing database driver errors and token library messages that help an attacker plan further attacks.

Remediation: The sensitive logging was removed. Handlers now log details on the server only and return generic messages, and a central error handler applies the same policy to unexpected errors.

V9) Weak password policy: Registration and doctor creation only required eight characters, so passwords such as `password` and `12345678` were accepted, which makes `bcrypt` hashing far less effective against guessing. The appointment cancellation handler also crashed with HTTP 500 when the appointment did not exist.

Remediation: Passwords must now contain at least eight characters including an uppercase letter, a lowercase letter, a digit and a symbol. NIST SP 800-63B favours longer passphrases checked against lists of breached passwords over composition rules [13]; we list this as future work in Section

V. The cancellation handler now handles missing appointments safely.

V10) Vulnerable third-party components: npm audit reported 17 advisories for the back end, including a critical issue in form-data and high-severity axios issues, and 22 advisories for each client, including a critical one.

Remediation: The dependencies were updated with npm audit fix, and `bcrypt` was upgraded from version 5 to 6, which removes a vulnerable install-time dependency chain. A misspelled Cloudinary variable name that silently disabled the media configuration was also corrected. The round 2 re-audit is discussed in Section IV-C.

B. Round 2 Findings

V11) Missing role enforcement and unpinned algorithm: The patient middleware accepted any valid token, so doctor and administrator tokens were treated as patient sessions. The doctor middleware rejected only tokens that carried a different role, so a token without a role claim was accepted. Controllers read the caller's identity from `req.body`, which is safe only as long as no later middleware rebuilds the body. `jwt.verify()` also accepted any HMAC algorithm.

Remediation: Each middleware now requires the exact role claim and a subject identifier, and answers HTTP 403 otherwise. The identity is attached to `req.auth` and never read from the request body, and verification accepts only HS256. Tests cover every cross-role combination.

V12) Upload processed before authentication: On `POST /api/user/update-profile`, the Multer middleware ran before the authentication middleware. Anonymous clients could therefore write files of up to 2 MB to the server's disk without any credentials, limited only by the general API rate limit. The round 1 type check trusted the client-declared MIME type and extension, so any content renamed to PNG was accepted. Temporary files were also never deleted after being sent to Cloudinary.

Remediation: The middleware order is now authentication, then Multer, then a content check that reads the file's first bytes and accepts only genuine JPEG, PNG and WEBP data. Temporary files are deleted after every request, and upload errors return HTTP 400. Tests confirm that an anonymous upload receives HTTP 401 without any file being written, and that a PHP script renamed to `.png` is rejected.

V13) Rate-limit bypass via spoofed X-Forwarded-For: Round 1 enabled Express's `trust proxy` setting unconditionally. When the API is reached directly, Express then takes the client IP address from the X-Forwarded-For header, which the attacker controls [14], and the rate limiter counts attempts per that value. Listing 3 reproduces the bypass: after ten attempts the real address was throttled, but twelve further attempts, each with a different header value, were never throttled.

```
same client address      400 x10, 429, 429
spoofed XFF per request  400 400 400 400 400 400 400
                        400 400 400 400 400 400
after the fix (spoofed)  400 x10, 429, 429
```

Listing 3. Status codes of 12 login attempts (V13).

Remediation: The proxy setting is now enabled only when `TRUST_PROXY` is configured. In the Docker deployment, nginx overwrites X-Forwarded-For with the real client address and the API has no published port, so trusting exactly one proxy is

safe there. This was verified both in-process and end-to-end through nginx.

V14) Account pre-hijacking and login oracles: Local registration does not verify that the user owns the email address, but the first Google sign-in implementation linked a Google identity to any existing account with the same address. An attacker could therefore register a victim's Gmail address with a password of their choosing. When the real owner later signed in with Google, the account was linked but the attacker's password still worked, giving the attacker lasting access. This is the account pre-hijacking pattern described by Sudhodanan and Pavard [15]. In addition, logging in to a Google-only account (which has no password hash) made bcrypt throw an exception, returning HTTP 500 instead of “Invalid Credentials!”. The handlers also skipped bcrypt for unknown email addresses, so both the status code and the response time revealed which accounts existed.

Remediation: When a Google identity is first linked to an existing account, the unverified local password is removed, the account is marked as a Google account, and all sessions issued before that moment are revoked through a new `sessionsValidAfter` timestamp. The patient middleware checks this timestamp, and also rejects tokens of deleted accounts. A new `verifyPassword()` function always performs exactly one bcrypt comparison (with a dummy hash when no hash exists) and never throws. In our measurement, logins for missing accounts took 58 ms and wrong passwords 56 ms. As a trade-off, users who registered locally continue with Google sign-in once they have used it.

V15) Weak administrator credential accepted: The administrator credential is read from environment variables, and nothing prevented a deployment from keeping the original weak password. The round 1 constant-time comparison also returned early when the lengths differed, which revealed the length of the password through response timing.

Remediation: `assertAdminCredentials()` stops the server at start-up unless the administrator email is valid and the password has at least 12 characters with mixed character classes. Both values are hashed with SHA-256 before a constant-time comparison.

V16) Double-booking race condition and booking logic flaws: The booking handler read the doctor's map of booked slots, checked availability in JavaScript, saved the appointment and then wrote the entire map back. Concurrent requests for the same slot all passed the check (a time-of-check to time-of-use race), and concurrent bookings of different slots could overwrite each other. The inputs were not validated: a malformed doctor identifier crashed the handler, past or impossible dates could be booked, and arbitrary strings, including `__proto__`, were used as map keys. Completed appointments could be cancelled, and cancelled ones completed. Repeated cancellations released the slot again, while a doctor's cancellation never released it at all. Appointment records also stored fields that other roles did not need, such as the doctor's email address and Google identifiers.

```
const doctor = await doctorModel.findOneAndUpdate({
  _id: docId, available: true,
  ['slots_booked.${slotDate}']: { $ne: slotTime } },
  { $push: { ['slots_booked.${slotDate}']: slotTime } });
if (!doctor) return "Slot not Available!";
```

Listing 4. Atomic check-and-reserve with a single conditional update.

Remediation: Listing 4 reserves a slot in one conditional update that matches only when the doctor is available and the slot is free. MongoDB applies it atomically to a single document, so only one concurrent request can succeed; if saving the appointment then fails, the slot is released again. Validators check identifiers, real calendar dates from yesterday up to 60 days ahead, and time formats. Status changes are atomic, with ownership included in the update filter, slots are released with an atomic removal, and appointment records store only the fields that are displayed. Against a real MongoDB 7 container, ten concurrent requests for the same slot produced exactly one booking and nine refusals.

V17) Mass assignment in profile updates: The doctor profile handler copied the fee, address and availability fields from the request straight into the database. A doctor could therefore set a negative or non-numeric fee, which becomes the amount charged for new appointments, or store arbitrary nested objects as the address. The patient profile handler parsed the address JSON without any checks, so malformed input caused HTTP 500, and it accepted any name, phone number, date of birth and gender. The add-doctor handler had the same address and fee problems.

Remediation: Only listed fields are accepted now, and each is validated: the address keeps only two text lines of at most 200 characters, fees must be finite numbers between 0 and 1,000,000, and phone numbers, dates of birth and gender values must match allowed formats or values. Invalid input receives HTTP 400.

V18) Replayable Google ID tokens: The first OpenID Connect implementation accepted any valid Google ID token issued for our client ID. No nonce was requested or checked, so an ID token captured anywhere (from browser history, a logging proxy, or a different login attempt) could be exchanged for an application session until it expired, typically after one hour. OpenID Connect Core defines the nonce parameter to bind an ID token to a client session and prevent replay [16]. The fix is described in Section VI.

V19) Credentials and personal data in browser consoles: Before submitting, the administrator panel printed every form field to the browser console, including the new doctor's plaintext password. The doctor login printed the session token, and the administrator, doctor and patient views printed complete API responses containing patients' names, telephone numbers, addresses and dates of birth. This data was visible to anyone with access to the workstation, a shared screen or a browser extension.

Remediation: The logging statements were removed. The fix also repaired the administrator and doctor login handler: misnamed state setters meant a successful login only took effect after a page reload, and an empty error handler silently ignored HTTP 401 and 429 responses.

V20) Rejected sessions not cleared by the clients: After a token expired or was revoked, the clients kept it in local storage. Every request failed, and the patient login page redirected any user who held a token, so the user could not sign in again without manually clearing browser storage.

Remediation: A shared axios response interceptor, installed when the application loads, clears the matching session on HTTP 401 and shows the API's generic error message instead of axios's default text.

C. Dependency Re-assessment

Table IV compares the two dependency audits. Within two months of the round 1 fix, new advisories affected all three packages: denial-of-service issues in brace-expansion and qs, incorrect address classification in ip-address (which can defeat SSRF protections) in the back end, and a CSRF bypass in React Router in both clients. The back end also still used the deprecated 1.x series of Multer, whose upload denial-of-service fixes exist only in version 2. All issues were resolved and Multer was upgraded to 2.4. This result motivates the weekly scheduled audit described in Section VII.

TABLE IV
NPM AUDIT RESULTS (ADVISORIES BEFORE AND AFTER EACH FIX)

Package	Jul. 2026	Sep. 2026 re-audit
backend	17 → 0	5 (2 high) → 0
frontend	22 → 0	2 (2 high) → 0
admin	22 → 0	2 (2 high) → 0

V. VULNERABILITIES NOT FIXED

Table V lists the weaknesses that remain, with the reason each was not fixed and the recommended action. Most require infrastructure (an email provider, a shared cache, logging services) or architectural changes that go beyond the scope of this assignment.

TABLE V
REMAINING WEAKNESSES

Issue	Reason not fixed	Recommendation
Leaked secrets remain in the original repository's history	Rotation is done in the provider dashboards, not in code	Rotate all keys; purge history with git filter-repo
JWTs stored in localStorage	httpOnly cookies need CSRF protection and CORS changes in three apps; partly mitigated by the CSP (Section VIII)	httpOnly, SameSite cookies with CSRF tokens
No logout endpoint or refresh-token rotation	Account-wide revocation exists; per-token revocation needs a token store	Short-lived access tokens, rotating refresh tokens
No email verification for local accounts	Requires an email provider; the linking attack (V14) is already closed	Verify ownership before activating an account
Single administrator from environment variables, no MFA	Needs a database-backed administrator model and an MFA flow	Role-based admin accounts with TOTP or WebAuthn
In-memory rate limits and nonce cache	Correct only while a single API instance runs	Shared Redis or MongoDB TTL store
Composition-based password rules	Simple to implement and explain	Passphrases and breached-password screening [13]
Razorpay payment not implemented	No server-side payment endpoint exists in the application	Server-side order creation and signature verification
No security event logging or alerting	Requires logging and monitoring infrastructure	Structured audit logs for logins and admin actions

VI. OPENID CONNECT SIGN-IN

A. Protocol Choice

OpenID Connect (OIDC) [16] adds an identity layer on top of OAuth 2.0 [17]: the identity provider issues a signed ID token that states who the user is. We used Google Identity Services [18]. Its Sign in with Google button returns an ID

token directly to the browser, which corresponds to the OIDC implicit flow with the id_token response type. The application uses Google only for authentication: it requests the openid, email and profile scopes, receives no Google access token, and therefore stores no refresh tokens or client secret. With this flow, the main risks are forged, misdirected or replayed ID tokens, and unsafe linking to existing accounts. The implementation addresses each of them.

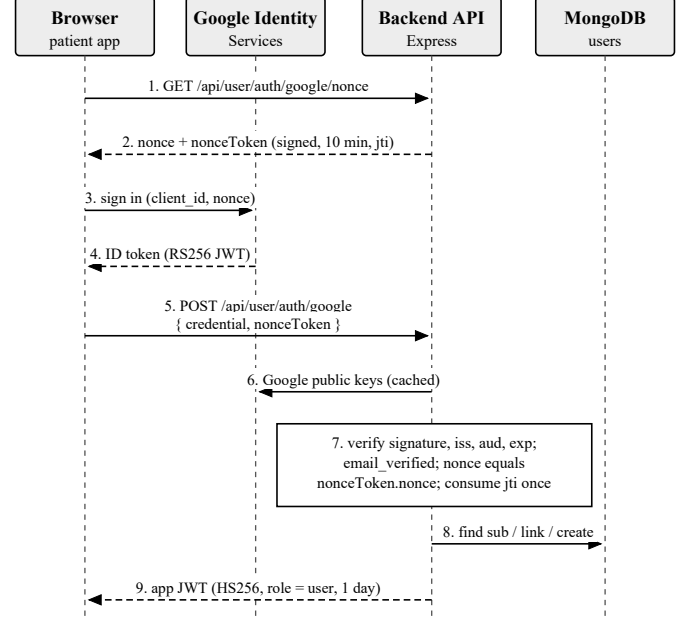


Fig. 3. OpenID Connect sign-in sequence with a server-issued, single-use nonce.

B. Sign-In Flow

Fig. 3 shows the message sequence. (1, 2) The patient application requests a nonce. The API generates a random 256-bit value and returns it together with a nonce token: a JWT with a unique identifier (jti), a ten-minute expiry, and a signature made with a key derived from the application secret by HMAC-SHA-256 with the label google-oidc-nonce. Because of this separate key, a nonce token can never be accepted as an access token. The response is marked as non-cacheable. (3, 4) The Google button is created with the nonce; after the user consents, Google returns an RS256-signed ID token containing that nonce. (5) The browser posts the ID token together with the nonce token. (6, 7) The API validates both, as described below. (8) It finds, links or creates the user, and (9) issues the application's own role-scoped JWT.

C. Server-Side Validation

Listing 5 shows the core checks. verifyIdToken from google-auth-library checks the RS256 signature against Google's published keys, the issuer, the audience (which must equal our client ID) and the expiry [19]. The API then requires a Google-verified email address, requires the token's nonce to equal the one in the nonce token, and marks the nonce token's jti as used, so a captured request cannot be replayed. Accounts are looked up by Google's stable subject identifier first and by email address second; linking follows the V14 rules.

```

const nonceClaims = verifyNonceToken(nonceToken);
const ticket = await googleClient.verifyIdToken({
  idToken: credential,
  audience: process.env.GOOGLE_CLIENT_ID });
const payload = ticket.getPayload();
if (payload?.nonce !== nonceClaims.nonce) return 401;
if (!consumeNonceId(nonceClaims.jti, nonceClaims.exp))
  return 401; // replay
if (!payload.email_verified) return 401;
// ...find, link or create the user by payload.sub
const token = signAccessToken({ id: user._id,
  role: "user" });

```

Listing 5. ID-token validation in the sign-in handler (abridged).

D. Client and Provider Configuration

The OAuth client in Google Cloud is a web application whose authorised JavaScript origins are the application's origins; users are restricted to registered test users while the app is in testing mode. The client ID is public and is built into the patient application; no client secret is used. The Google button is rendered only once a nonce is available, and it is recreated with a fresh nonce after every attempt. The nginx headers are compatible with the provider: the Content Security Policy allows Google's sign-in script, frame and style origins, Cross-Origin-Opener-Policy allows the sign-in popup, and the Referrer-Policy keeps the origin information that Google requires.

E. Verification

Automated tests cover the following cases: nonces are random and non-cacheable; a missing nonce token is rejected with HTTP 400; a nonce token used as an access token is rejected; an ID token issued for a different nonce is rejected; linking removes a pre-set password and revokes earlier sessions; and replaying a successful request fails. In manual tests with real Google accounts, sign-in, account provisioning and profile loading worked end to end.

VII. VERIFICATION AND CONTINUOUS SECURITY

A. Automated Regression Tests

To make the API testable, the Express application was separated (`app.js`) from start-up, which loads configuration, performs the secret checks and connects to the database (`server.js`). The suite uses Node's built-in test runner and supertest. It runs against the API in the same process with database calls replaced by stubs, so it needs no database or external accounts. It contains 41 tests grouped as shown in Table VI.

TABLE VI
VERIFICATION SUMMARY

Suite	Content	Result
npm test	HTTP hardening (3), JWT and roles (11), start-up secret checks (2), injection and login oracles (2), uploads (5), input validation (4), OIDC (5), rate limiting (1), validators and password helper (8)	41 / 41 pass
End-to-end	23 checks through nginx against MongoDB 7, including the concurrent booking race, patient, doctor and admin flows, and spoofed-header throttling	23 / 23 pass
npm audit	backend, frontend, admin; high and critical advisories	0 found
CI	GitHub Actions and CodeQL on every push, pull request and weekly	Passing

B. End-to-End Test

A script exercised the Docker deployment through the nginx proxies. It confirmed the security headers and single-page-application routing, that port 4000 is not published, that foreign origins are rejected, and the patient registration, profile, booking and cancellation flows, doctor and administrator logins, and dashboards. It also confirmed exactly-once booking under ten concurrent requests, slot release on cancellation, refusal of a second cancellation, minimal appointment records and throttling despite spoofed headers. The script removes its test data afterwards.

C. Continuous Security Pipeline

A GitHub Actions workflow runs on every push, on every pull request and weekly. It executes the regression tests, fails the build on any high or critical npm advisory in the three packages, builds both clients and scans pushed commits for credentials with gitleaks. A separate workflow runs CodeQL with the security-extended query suite [9], and Dependabot proposes dependency updates weekly. At the time of writing, both workflows passed on the main branch.

VIII. SECURE DEPLOYMENT

The system is packaged as three images plus MongoDB 7 and orchestrated with Docker Compose (Fig. 1), following the OWASP Docker security guidance [20]. The API image is based on Node.js 22 Alpine, contains only production dependencies and runs as a non-root user. Each client is built with Vite and served by an unprivileged nginx image, which also proxies API requests, so browsers talk to a single origin.

- *Network exposure*: only the two web containers publish ports. The API and MongoDB are reachable only inside the Compose network, and nginx overwrites X-Forwarded-For, so the rate limiter always sees the real client address (V13).
- *Least privilege*: the API runs as UID 1000 and nginx as UID 101. The application containers have read-only root filesystems, drop all Linux capabilities and set no-new-privileges; only /tmp and the upload directory are writable, as in-memory filesystems.
- *Browser policies*: nginx sends a Content Security Policy [21] that allows scripts only from the application's own origin, Google's sign-in service and Razorpay, and forbids plugins, framing and foreign form targets. Together with X-Frame-Options, nosniff, Referrer-Policy and Permissions-Policy, this reduces the impact of cross-site scripting on tokens stored in local storage.
- *Secrets and data*: MongoDB requires authentication and stores data in a named volume. Secrets come from git-ignored environment files and are excluded from images, and health checks enforce the start-up order (database, then API, then web servers).

IX. PREVENTIVE BEST PRACTICES

Each vulnerability corresponds to a practice of the NIST Secure Software Development Framework [22] and established security development lifecycles [23], [24] that was missing:

- *Threat modelling and abuse cases* for authentication, social-login account linking and concurrent booking would have exposed V2, V14 and V16 at design time.

- *Secure defaults and shared security components*, namely one token module, one validation layer and a hardened Express template, prevent the inconsistent checks behind V3, V5, V11 and V17.
- *Secrets management*: environment templates, a secrets manager, and secret scanning before commits and in CI address V1 and V15.
- *Checklist-based code review* (for example with ASVS) that covers middleware order, logging of sensitive data and client-side logging would have caught V8, V12 and V19.
- *Continuous security testing*: static analysis, dependency scanning, dynamic testing and a regression test for every fixed vulnerability detect V4, V6 and V10 early and keep fixes from regressing.
- *Reviewing security fixes themselves*: V12 and V13 remained after, or were introduced by, round 1 hardening and were found only by a second independent review.

X. INDIVIDUAL CONTRIBUTIONS

Table VII lists each member's primary responsibilities.

TABLE VII
INDIVIDUAL CONTRIBUTIONS

Member	Area	Work items
Nethmina W.P.R. (IT22253958)	Authentication and session security	V2, V3, V11, V15, V20; repository setup and CI pipeline
Appuhami M.N.H. (IT22140852)	Injection, input validation and uploads	V6, V7, V12, V16, V17
Madurapperuma H.A.S.I. (IT22230942)	Configuration, secrets, dependencies and deployment	V1, V4, V5, V10, V13; Docker deployment
Aron Charles J. (IT22203380)	OpenID Connect and data protection	Google sign-in, V8, V9, V14, V18, V19

XI. CONCLUSION

We assessed and hardened Prescripto, a MERN-stack doctor appointment booking system. Two rounds of review identified 20 distinct vulnerabilities across nine OWASP Top 10:2021 categories, ranging from committed secrets and forgeable administrator tokens to account pre-hijacking, a race condition in booking and a rate-limit bypass introduced by an earlier fix. All were remediated in documented commits and are protected against regression by 41 automated tests, a 23-check end-to-end test and a continuous security pipeline. Google Sign-In based on OpenID Connect was added with complete server-side validation, single-use nonces and safe account linking, and the system now runs in hardened containers. Future work includes cookie-based sessions with CSRF protection, multi-factor authentication for administrators, email verification, breached-password screening, and dynamic scanning (for example with OWASP ZAP) and container image scanning in the pipeline. The source code and commit history are available in the project repository [4].

REFERENCES

- [1] OWASP Foundation, "OWASP Top 10:2021," 2021. Accessed: Sep. 2026. [Online]. Available: <https://owasp.org/Top10/>
- [2] The MITRE Corporation, "Common Weakness Enumeration (CWE)." Accessed: Sep. 2026. [Online]. Available: <https://cwe.mitre.org/>
- [3] RVNethmina, "AppointmentBookingSystem," GitHub repository, baseline commit 2c2bad8, Feb. 2025. [Online]. Available: <https://github.com/RVNethmina/AppointmentBookingSystem>
- [4] RVNethmina, "SE4030-secure-appointment-system," GitHub repository, Sep. 2026. [Online]. Available: <https://github.com/RVNethmina/SE4030-secure-appointment-system>
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
- [6] OWASP Foundation, "Application Security Verification Standard 4.0.3," Oct. 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>
- [7] npm, Inc., "npm-audit," npm Docs. Accessed: Sep. 2026. [Online]. Available: <https://docs.npmjs.com/cli/commands/npm-audit>
- [8] Gitleaks, "Gitleaks," GitHub repository. Accessed: Sep. 2026. [Online]. Available: <https://github.com/gitleaks/gitleaks>
- [9] GitHub, Inc., "About code scanning with CodeQL," GitHub Docs. Accessed: Sep. 2026. [Online]. Available: <https://docs.github.com/en/code-security/code-scanning>
- [10] FIRST, "Common Vulnerability Scoring System v3.1: Specification document," Jun. 2019. [Online]. Available: <https://www.first.org/cvss/v3.1/specification-document>
- [11] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token best current practices," IETF RFC 8725, Feb. 2020.
- [12] OWASP Foundation, "Testing for NoSQL injection," in *OWASP Web Security Testing Guide*, v4.2, Dec. 2020. [Online]. Available: <https://owasp.org/www-project-web-security-testing-guide/>
- [13] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, Jun. 2017.
- [14] Express, "Express behind proxies," Express documentation. Accessed: Sep. 2026. [Online]. Available: <https://expressjs.com/en/guide/behind-proxies.html>
- [15] A. Sudhodanan and A. Paverd, "Pre-hijacked accounts: An empirical study of security failures in user account creation on the web," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, Aug. 2022.
- [16] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 2," OpenID Foundation, Dec. 2023. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [17] D. Hardt, Ed., "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012.
- [18] Google, "Sign in with Google for web," Google for Developers. Accessed: Sep. 2026. [Online]. Available: <https://developers.google.com/identity/gsi/web>
- [19] Google, "Verify the Google ID token on your server side," Google for Developers. Accessed: Sep. 2026. [Online]. Available: <https://developer.s.google.com/identity/gsi/web/guides/verify-google-id-token>
- [20] OWASP Foundation, "Docker security cheat sheet," OWASP Cheat Sheet Series. Accessed: Sep. 2026. [Online]. Available: https://cheatsheets.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- [21] W3C, "Content Security Policy Level 3," W3C Working Draft. Accessed: Sep. 2026. [Online]. Available: <https://www.w3.org/TR/CSP3/>
- [22] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," NIST Special Publication 800-218, Feb. 2022.
- [23] M. Howard and S. Lipner, *The Security Development Lifecycle*. Redmond, WA, USA: Microsoft Press, 2006.
- [24] G. McGraw, *Software Security: Building Security In*. Boston, MA, USA: Addison-Wesley, 2006.